08 — One-Page Cheat Sheet (review the night before + morning of the test)

Scoring: 600 marks total, pass = 350. Partial credit is real — never leave a question blank. Lock Q1 + Q2 + Q3. Q4 is a bonus.

Q	What it really is	Attack plan
Q1	Simple loop / math / condition	Just code it. `%`, `//`, `min/max/sum`. 5–10 min.
Q2	Lists & strings	Slicing, `.split()`, `.join()`, `sorted(key=...)`.
Q3	Long description, follow rules EXACTLY	Read **twice**. Rules as comments first. Hand-trace the example.
Q4	Needs one efficient idea	See the recognition table. If n ≤ 1000 → brute force is fine.

PART 1 — THE 6 LOOP SHAPES (every problem is one of these)

Ask first: do I know the answer after looking at ONE item? Yes → SEARCH (return inside). No → everything else (**return AFTER the loop**).

```python
# 1. TRANSFORM — change every item
result = []
for x in items:
    result.append(x * 2)
return result

# 2. FILTER — keep only some
result = []
for x in items:
    if x > 0:
        result.append(x)
return result

# 3. REDUCE — one number out (sum / count)
total = 0                    # count = 0
for x in items:
    total += x               # count += 1   <- COUNT adds 1, SUM ad
return total

# 4. BEST-SO-FAR — the biggest/longest one
best = items[0]
for x in items:
    if x > best:             # or len(x) > len(best)
        best = x             # plain '=', never '+='
return best

# 5. SEARCH — find one thing, stop early   (return INSIDE!)
for x in items:
    if x < 0:
        return x             # found it
return 0                     # fallback: never found

# 6. COUNT EACH — how many of every value
counts = {}
for x in items:
    counts[x] = counts.get(x, 0) + 1
return counts
```

Two-phase problems (very common) — one shape, then another

```
# "all the longest words" = BEST-SO-FAR, then FILTER
longest = 0
for w in words:
    if len(w) > longest:
        longest = len(w)
result = []
for w in words:
    if len(w) == longest:
        result.append(w)
return result

# "most frequent value" = COUNT EACH, then BEST-SO-FAR (track TWO va
counts = {}
for x in nums:
    counts[x] = counts.get(x, 0) + 1
best_key, best_count = None, 0
for key, value in counts.items():
    if value > best_count:
        best_count = value       # the count
        best_key = key           # AND which key had it
return best_key
```

PART 2 — NUMBERS

```
a + b   a - b   a * b   a ** b     # add, subtract, multiply, power
a / b        # 7 / 2  = 3.5   ALWAYS a float
a // b       # 7 // 2 = 3     integer divide  <- usually what you wa
a % b        # 7 % 2  = 1     remainder

abs(-5)      # 5      distance from zero
round(3.7)   # 4
min(4, 7)    max(4, 7)
int("42")    str(42)    float("3.5")     # converting
```

% and // — the exam-gold table

I want	Use	Example
last digit	`n % 10`	`1234 % 10` = 4
remove last digit	`n // 10`	`1234 // 10` = 123
is even / odd	`n % 2 == 0` / `== 1`	
divisible by k	`n % k == 0`	

wrap around a cycle of k	`i % k`	alphabet: `% 26`
how many whole groups	`total // k`	
divide rounding UP	`(x + k - 1) // k`	century: `(year + 99) // 100`

Digits of a number

```
# PEEL them off (right to left)
while n > 0:
    digit = n % 10
    n = n // 10

# or turn it into a string
[int(d) for d in str(n)]      # 1234 -> [1,2,3,4]
str(n)[::-1]                  # reversed digits as text
int(str(n)[::-1])             # reverse a number: 1234 -> 4321
```

> **GENERATE vs PEEL:** "numbers 1 to n" → `range(1, n+1)`. "digits of n" → `% 10` / `// 10`. A calculation on n → no loop at all.

NUMBER PATTERNS — full skeletons

N1 — "sum / count the numbers 1 to n that ___" · *GENERATE with range*

```
total = 0
for i in range(1, n + 1):           # n+1 so n itself is included
    if i % 2 == 1:                  # the condition
        total += i
return total
```

N2 — "sum of the digits" · *PEEL the digits*

```
total = 0
while n > 0:
    total += n % 10                 # grab the last digit
    n = n // 10                     # drop it
return total
```

Shortcut: `return sum(int(d) for d in str(n))`

N3 — "reverse the number" · *1234 -> 4321*

```
result = 0
while n > 0:
    result = result * 10 + n % 10   # shift left, drop the digit in
    n = n // 10
return result
```

Shortcut: `return int(str(n)[::-1])`

N4 — "how many digits do ___" · *digits that divide / are even / etc.*

```
count = 0
original = n                        # SAVE it — the loop destroys n
while n > 0:
    d = n % 10
    if d == 0:                      # guard: never divide by zero
        pass
    elif original % d == 0:         # the condition, tested against
        count += 1
    n = n // 10
return count
```

N5 — "is it leap / prime / valid" · *rule-following predicate*

```
if year % 400 == 0:                 # STRONGEST exception first
    return True
if year % 100 == 0:
    return False
return year % 4 == 0                # the comparison IS True/False
```

N6 — "which century / how many boxes of k" · *divide, rounding UP*

```
return (x + k - 1) // k             # century: (year + 99) // 100
```

N7 — "wrap around a cycle" · *alphabet shift / go round the list*

```
new_index = (i + shift) % k         # k = 26 for the alphabet
```

> **No loop needed** when it's a single calculation on one number (`century`, `is_leap`). Before writing a loop, name what it walks over — if you can't, you don't need one.

PART 3 — STRINGS

```
 a   m   a   l   i
 0   1   2   3   4      from the front
-5  -4  -3  -2  -1      from the back
```

```
s[0]  s[-1]        # first / last character
s[1:4]             # from 1, STOPS BEFORE 4
s[:3]   s[2:]      # first three / from 2 to the end
s[::-1]            # REVERSE
s[::2]             # every 2nd character
len(s)             # how many characters
```

```
s.upper()   s.lower()          # change case
s.strip()                      # remove spaces at both ends
s.split()                      # "a b c" -> ["a","b","c"]
s.split(",")                   # split on a comma
"-".join(list_of_strings)      # ["a","b"] -> "a-b"
s.count("a")                   # how many "a"
s.replace("a", "o")            # NEW string with swaps
s.find("li")                   # position, or -1
s.startswith("am")  s.endswith("ch")
c.isdigit()  c.isalpha()  c.islower()  c.isupper()
ord('a')   # 97      chr(98)   # 'b'
"cat" in sentence              # substring present?
for ch in s:                   # walk the characters
```

Strings are IMMUTABLE — `s[0] = "X"` crashes. To change one: `lst = list(s); lst[0] = "X"; s = "".join(lst)`

String one-liners

```
s == s[::-1]                    # palindrome
sorted(a) == sorted(b)          # anagram
"".join(sorted(word))           # letters in order: "cab" -> "abc
s.lower().replace(" ", "")      # normalise before comparing
" ".join(s.split()[::-1])       # reverse the WORD order
max(s.split(), key=len)         # longest word
```

STRING PATTERNS — full skeletons

S1 — "count the vowels / digits / spaces" · *how many ___ are in the string*

```
count = 0
for ch in s:
    if ch in "aeiou":               # the condition
        count += 1
return count
```

S2 — "keep only the letters / remove the vowels" · *filter characters*

```
result = []                         # a LIST, because strings are i
for ch in s:
    if ch.isalpha():                # the keep-condition
        result.append(ch)
return "".join(result)              # fuse back into a string at th
```

S3 — "uppercase / reverse every word" · *change each word, keep the sentence*

```
words = s.split()
result = []
for w in words:
    result.append(w.upper())        # whatever the change is
return " ".join(result)
```

S4 — "first character that is a digit" · *first letter/char that ___*

```
for ch in s:
    if ch.isdigit():                # the find-condition
        return ch                   # SEARCH — return inside
return ""                           # fallback: none found
```

S5 — "the longest word" · *longest / shortest word in a sentence*

```
words = s.split()
best = words[0]
for w in words:
    if len(w) > len(best):          # compare LENGTHS, store the WO
        best = w
return best
```

S6 — "count each letter" · *letter frequency / how many of each character*

```
counts = {}
for ch in s:
    counts[ch] = counts.get(ch, 0) + 1
return counts
```

S7 — "first non-repeating character" · *first letter appearing exactly once* → S6 first, then SEARCH the original order

```
counts = {}
for ch in s:
    counts[ch] = counts.get(ch, 0) + 1

for ch in s:                          # re-walk the ORIGINAL string,
    if counts[ch] == 1:
        return ch
return ""
```

S8 — "is it a palindrome / anagram" · *reads the same both ways / same letters*

```
return s == s[::-1]                               # palindrome
return sorted(a) == sorted(b)                     # anagram
return s.lower().replace(" ", "") == s.lower().replace(" ", "")[::-
```

S9 — "reverse / rotate / take part of it" · *slicing, no loop needed*

```
s[::-1]              # reverse the characters
" ".join(s.split()[::-1])     # reverse the WORD order
s[k:] + s[:k]        # rotate left by k
s[1:-1]              # drop first and last character
```

> **Building a string?** Collect pieces in a **list**, then `"".join()` at the end — strings are immutable, so you can never append to one.

PART 4 — LISTS

```
nums[0]  nums[-1]   nums[1:4]   nums[::-1]   nums[::2]    # same sl
len(nums)                       # how many items
nums.append(5)                  # add to end          (returns None
nums.pop()      nums.pop(0)     # remove last / first
nums.insert(1, 99)              # insert at position
nums.index(4)                   # position of first 4, crashes if al
nums.remove(3)                  # delete first 3 BY VALUE
nums.count(1)                   # how many 1s
5 in nums                       # membership
nums2 = nums[:]                 # a real COPY (b = a does NOT copy)
a + b                           # join two lists
[0] * 5                         # [0,0,0,0,0]
list(range(5))                  # [0,1,2,3,4]
sorted(nums)                    # NEW sorted list
sorted(nums, reverse=True)      # biggest first
sum(nums)  min(nums)  max(nums)
max(words, key=len)             # longest item — key= works in max/
```

List one-liners

```
[x * 2 for x in nums]              # transform
[x for x in nums if x > 0]         # filter
[len(w) for w in words]            # lengths
any(x < 0 for x in nums)           # is ANY negative?
all(x > 0 for x in nums)           # are ALL positive?
nums[::2]   nums[1::2]             # even / odd positions
nums[k:] + nums[:k]                # rotate left by k
sorted(nums, reverse=True)[:3]     # 3 largest
```

LIST PATTERNS — full skeletons

L1 — "change every item" · *double / square / uppercase each / lengths of*

```
result = []
for x in nums:
    result.append(x * 2)            # whatever the change is
return result
```

L2 — "keep only the ones that ___" · *only the / filter out / remove the*

```
result = []
for x in nums:
    if x > 0:                          # the keep-condition
        result.append(x)
return result
```

L3 — "total / how many satisfy ___" · *sum of / count of / how many are*

```
total = 0                              # count = 0
for x in nums:
    if x % 2 == 0:                     # drop the if when it's ALL iter
        total += x                     # SUM adds x  ·  COUNT adds 1
return total
```

L4 — "the largest / longest item" · *biggest / maximum / longest*

```
best = nums[0]                         # start at the first item, not 0
for x in nums:
    if x > best:                       # or len(x) > len(best) for word
        best = x                       # plain '=', never '+='
return best
```

L5 — "the first one that ___" · *first / find / does it contain*

```
for x in nums:
    if x < 0:                          # the find-condition
        return x                       # SEARCH — return inside
return 0                               # fallback: never found
```

L6 — "ALL the longest / everything tied for max" · *all the / every one that is most* → L4 first, then L2

```
longest = 0
for w in words:
    if len(w) > longest:
        longest = len(w)               # phase 1: find the target

result = []
for w in words:
    if len(w) == longest:              # phase 2: collect ALL matches
        result.append(w)
return result
```

L7 — "move zeros / evens to the end" · *move ___ to the end / group ___ first*

```
keep = []
move = []
for x in nums:
    if x == 0:
        move.append(x)
    else:
        keep.append(x)
return keep + move                  # two lists, joined with +
```

L8 — "remove duplicates, keep order" · *unique / distinct, original order*

```
seen = set()
result = []
for x in nums:
    if x not in seen:
        result.append(x)
        seen.add(x)
return result
```

L9 — "compare each item to the NEXT" · *adjacent / neighbouring / consecutive pairs*

```
result = []
for i in range(len(nums) - 1):      # -1 so nums[i+1] always exists
    result.append(nums[i] + nums[i + 1])
return result
```

L10 — "two lists side by side" · *pair up / match position by position*

```
result = []
for a, b in zip(list1, list2):
    result.append(a + b)
return result
```

L11 — "even / odd positions" · *alternating / every other / two teams*

```
return [sum(nums[::2]), sum(nums[1::2])]     # [::2] = positions 0,
```

> **Rearranging (move/remove/reorder)?** Don't edit in place — **build a new list** in the order you want, then return it. That turns it into L1 or L2.

PART 5 — DICTIONARIES & SETS

```
d = {"ama": 20}
d["ama"]                # look up (CRASHES if missing)
d.get("zoe", 0)         # look up with a default — safe
d["yaw"] = 30           # add or overwrite
"ama" in d              # does this KEY exist?
len(d)                  # how many pairs
for k in d:             # keys only
for k, v in d.items():  # key AND value
list(d.values())        # all values as a list
list(d.keys())          # all keys as a list

from collections import Counter
Counter("aba")               # {'a':2,'b':1}
Counter(nums).most_common(1) # [(value, count)]



seen = set()            # empty set  ({} is an empty DICT!)
seen.add(5)             # (returns None!)
5 in seen               # instant check
set(nums)               # remove duplicates
len(set(nums))          # how many distinct
sorted(set(a) & set(b)) # in BOTH, sorted
set(a) | set(b)         # in either
set(a) - set(b)         # in a but not b
```

> **Speed:** `x in list` re-scans everything (slow in a loop). `x in set` / `x in dict` is instant. That swap is often what makes a slow Q4 pass.

DICT / SET PATTERNS — full skeletons

D1 — "how many of each" · *count / frequency / tally / how many times*

```
counts = {}
for x in items:
    counts[x] = counts.get(x, 0) + 1
return counts
```

D2 — "most frequent" · *most common / appears most / highest count* → D1 first, then best-so-far over `.items()` with **two** variables

```
counts = {}
for x in nums:
    counts[x] = counts.get(x, 0) + 1

best_key = None
best_count = 0
for key, value in counts.items():
    if value > best_count:
        best_count = value          # the count
        best_key = key              # AND which key had it
return best_key                     # the KEY, not the count
```

D3 — "first duplicate" · *first repeat / appears a second time / seen before*

```
seen = set()
for x in nums:
    if x in seen:
        return x                    # SEARCH — return inside
    seen.add(x)                     # only runs if not returned
return -1                           # fallback: no duplicate
```

D4 — "are there any duplicates" · *all unique? / any repeats?*

```
return len(set(nums)) != len(nums)
```

D5 — "how many distinct" · *different / unique values*

```
return len(set(nums))
```

D6 — "in both lists" · *common / shared / appear in both*

```
return sorted(set(a) & set(b))      # & = both   | = either   - = i
```

D7 — "pair summing to target" · *two numbers add up to / find its partner*

```
seen = set()
for x in nums:
    if target - x in seen:          # has my partner gone by alread
        return True
    seen.add(x)
return False
```

D8 — "group by ___" · *put the same ___ together / buckets / anagram groups*

```
groups = {}
for w in words:
    key = ''.join(sorted(w))                # the LABEL that matche
    groups.setdefault(key, []).append(w)
return list(groups.values())
```

D9 — "first appearing exactly once" · *first non-repeating / first unique* → D1 first, then SEARCH the **original** order

```
counts = {}
for ch in s:
    counts[ch] = counts.get(ch, 0) + 1

for ch in s:                            # walk the ORIGINAL again — that
    if counts[ch] == 1:
        return ch
return ""
```

D10 — "can I build A from B's letters" · *ransom note / enough letters*

```
from collections import Counter
need = Counter(a)
have = Counter(b)
for ch in need:
    if have[ch] < need[ch]:
        return False
return True
```

> **Count-then-decide:** anything depending on totals needs **two passes** — count everything first, then walk again to decide. You cannot judge on pass one.

PART 6 — GRIDS (Q3)

```
rows = len(grid)          cols = len(grid[0])
grid[r][c]                # ROW first, then COLUMN

for r in range(rows):     # visit every cell
    for c in range(cols):
        ...grid[r][c]...

grid = [[0] * cols for _ in range(rows)]   # correct build
grid = [[0] * cols] * rows                 # BUG — shared rows!

col = [grid[r][j] for r in range(rows)]    # column j

# neighbours
directions = [(-1,0),(1,0),(0,-1),(0,1)]                        # 4-
directions = [(-1,-1),(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0),(1,1)
for dr, dc in directions:
    nr, nc = r + dr, c + dc
    if 0 <= nr < rows and 0 <= nc < cols:   # BOUNDS CHECK FIRST
        ...grid[nr][nc]...
```

GRID PATTERNS — full skeletons

G1 — "the value at row r, column c" · *no loop, direct access*

```
return grid[r][c]                    # ROW first, then COLUMN
```

G2 — "how big is the grid" · *dimensions / size*

```
rows = len(grid)                     # how many rows
cols = len(grid[0])                  # how many columns (length of O
return [rows, cols]
```

G3 — "get column j" · *a whole column / vertical line*

```
result = []
for r in range(len(grid)):
    result.append(grid[r][j])        # r MOVES, j stays FIXED
return result
```

(A **row** needs no loop — `grid[i]` is already a list.)

G4 — "the diagonal" · *top-left to bottom-right*

```
result = []
for i in range(len(grid)):
    result.append(grid[i][i])       # both move together
return result
```

G5 — "do something to every cell" · *sum all / count / find biggest*

```
total = 0
for r in range(len(grid)):
    for c in range(len(grid[0])):
        total += grid[r][c]         # or: if cond: count += 1
return total
```

G6 — "build a new grid" · *double every value / transform each cell*

```
result = []
for r in range(len(grid)):
    new_row = []                    # a fresh list PER ROW
    for c in range(len(grid[0])):
        new_row.append(grid[r][c] * 2)
    result.append(new_row)          # add the finished row
return result
```

G7 — "make an empty grid" · *rows x cols filled with a value*

```
grid = [[value] * cols for _ in range(rows)]     # correct
grid = [[value] * cols] * rows                   # BUG — shared rows
```

G8 — "sum/check each row" · *row totals / is any row all zeros*

```
result = []
for row in grid:                    # each row is already a list
    result.append(sum(row))
return result
```

G9 — "sum/check each column" · *column totals*

```
result = []
for c in range(len(grid[0])):       # outer loop = COLUMN this time
    total = 0
    for r in range(len(grid)):
        total += grid[r][c]
    result.append(total)
return result
```

G10 — "the neighbours of a cell" · *adjacent / surrounding / Minesweeper*

```
directions = [(-1,0),(1,0),(0,-1),(0,1)]        # 4-way
# 8-way adds: (-1,-1),(-1,1),(1,-1),(1,1)
count = 0
for dr, dc in directions:
    nr, nc = r + dr, c + dc                      # new row, new colu
    if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]):    # BOUNDS
        count += grid[nr][nc]
return count
```

G11 — "walk a column until something stops you" · *matrixElementsSum*

```
total = 0
for c in range(len(grid[0])):
    for r in range(len(grid)):
        if grid[r][c] == 0:
            break                    # stop THIS column, move to the
        total += grid[r][c]
return total
```

> **Row loop vs column loop:** `for r ... for c` sweeps left-to-right, row by row. Swap them (`for c ... for r`) to go top-to-bottom, column by column. The order matters when you `break`.

PART 7 — PHRASE → TOOL

The problem says...	Use	Code seed
"count / how many times / most frequent"	dict or Counter	`counts[x] = counts.get(x,0)+1`
"duplicate / seen before / unique"	set	`if x in seen: ... seen.add(x)`

"pair that sums to target"	set (Two Sum)	`if target - x in seen:`
"group by / same ___ together"	dict of lists	`groups.setdefault(k, []).append(x)`
"reverse"	slicing	`s[::-1]`
"digits of a number"	`%10` / `//10`	or `[int(d) for d in str(n)]`
"every 2nd / alternating"	step slicing	`a[::2]`, `a[1::2]`
"biggest / best so far"	running best	shape 4 above
"longest word / sort by size"	`key=len`	`max(words, key=len)`
"neighbours in a grid"	direction list	+ bounds check
"min difference / closest"	sort first	then compare neighbours
"sum of a range / subarray"	prefix sums	`prefix.append(prefix[-1]+x)`
"longest CONTIGUOUS chunk"	sliding window	grow right, shrink left
"n ≤ 1000"	brute force	double loop is fine

PART 8 — TRAPS THAT COST MARKS

> ### ⚠️ THE ONE THAT KEEPS CATCHING ME: =+ vs +=
>
> ```
> x += 5     # ✓ x is 15   — ADD (plus FIRST, touching the variab
> x =+ 5     # ✗ x is 5    — means x = +5, a plain assignment
> ```
>
> **Python gives no error.** It runs and silently returns the wrong answer. Every time I type =, check whether a + belongs before it.

> ### ⚠️ range NEEDS A NUMBER — so range(len(...))
>
> ```
> for r in range(len(grid)):     # ✓ len needed
> for r in range(grid):          # ✗ CRASH — a list is not a numb
> ```

> **`range(len(` is ONE unit.** Nowhere else needs `len()`: `grid[r][c]`, `grid[0]`, `nums[-1]`, `for x in nums` — all fine without it.

- **`return`, don't `print`.** The grader reads the return value.
- **`return` inside a collecting loop** → only the first item processed. After the loop!
- **Setup inside the loop** (`result = []`) → resets every turn. Before the loop!
- **`.append()` `.sort()` `.add()` return `None`** — call them, never assign them.
- **`sorted(a)` wraps; `a.sort()` attaches.** There is no `a.sorted()`.
- **`len(list)` vs `len(item)`** — how many, vs how long.
- **`=+` is not `+=`** — `x =+ 5` means `x = +5`, silently wrong.
- **`b = a` does NOT copy a list** → `b = a[:]`.
- **`[[0]*c]*r` shares rows** → use the comprehension.
- **Strings are immutable** — build with a list + `"".join()`.
- **`range(n)` stops at `n−1`** — for neighbours loop `range(len(a) - 1)`.
- **`/` gives a float, `//` gives an integer.**
- **Dividing or `%` by zero crashes** — guard it first.
- **`{}` is an empty dict**, not a set. Use `set()`.
- **`d[missing]` crashes** — use `d.get(key, default)`.

PART 9 — EXAM-DAY RULES

1. **Set the language to Python 3** before you start.
2. Scan all 4 questions in the first minute. Start with the one you can solve fastest.
3. ~**17 minutes per question.** Watch the clock.
4. **RUN TESTS early and often** — after every few lines, not at the end.
5. Q3: read twice, write the rules as comments, hand-trace the example.
6. Test edge cases: empty, one element, all-same, 1×1 grid.
7. If n ≤ 1000, brute force. Correct-but-slow beats clever-but-broken.
8. Stuck 10+ min? You're overcomplicating. Re-read; simpler idea.
9. **Fully complete two questions before polishing the rest.**
10. **Never leave a blank** — partial credit is real. Submit whatever passes.

PART 10 — COGNITIVE TEST

- Sequences: check +d, growing +d, ×k, alternating, squares/primes — in that order.
- x% of y = y% of x (8% of 25 = 25% of 8 = 2).
- 10% = ÷10; 5% = half of that; 1% = ÷100 — build any % from these.
- Ratio 3:2 with total 40 → parts of 8 → 24 and 16.
- "True / False / Cannot say": use ONLY the passage. Not stated → **Cannot say**.
- 30–60 sec per question max; accuracy over rushing; guess rather than leave blank.